

MÉMOIRE TECHNIQUE

IfSecurity

Plateforme de mitigation des attaques par brute-force

SSH · Web · Détection & réponse active

Auteur : Mohamed Youssef Jouini

Formation : Cycle Ingénieur — UTT — Réseaux & Télécoms / Sécurité des Systèmes

Date : Juin 2026

Version : v2 — Détection & réponse active

1

Résumé exécutif

IfSecurity est une plateforme conteneurisée de défense en profondeur contre les attaques par **brute-force**, sur deux surfaces simultanément : l'authentification **SSH** et un portail **web** Flask. Elle combine durcissement protocolaire, authentification forte, observabilité structurée et — c'est l'apport central de la v2 — une **mitigation active** qui ferme la boucle détection → réponse en bannissant automatiquement les IPs attaquantes au niveau réseau via `iptables`.

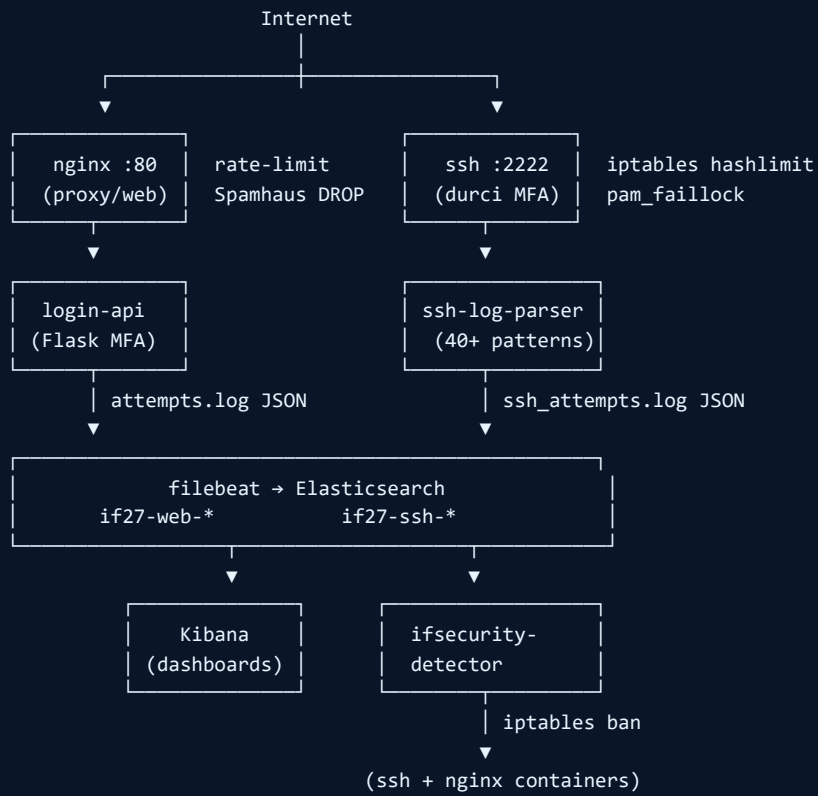
Mécanismes implémentés (vue d'avion)

- **5 couches de défense** : réseau (rate-limit, threat intel) → protocole (MFA, logout) → applicatif (rate-limit, reCAPTCHA) → observabilité (parsing, Elasticsearch, Kibana) → réponse (ban actif).
- **Détection** en temps réel des bursts d'attaques (parser SSH custom, agrégateur anti-bruit).
- **Mitigation active** centralisée via un détecteur qui consomme Elasticsearch et applique `iptables DROP` sur les containers exposés.
- **Honeypot** optionnel (endlesssh) pour ralentir et instrumenter les attaquants.

2

Architecture

La plateforme est déployée par `docker compose` en 9 services. Les flux de données (logs, événements, ordres de bannissement) sont représentés ci-dessous.



3

Synthèse des solutions implémentées

Couche	Mécanisme	Fichier / Composant	Statut
Réseau	Rate-limit iptables (hashlimit 3 conn/min/IP)	ssh/entrypoint.sh	actif
	Threat intel Spamhaus DROP	ssh/entrypoint.sh	actif
SSH	Auth forte 3 modes (A/B/C)	ssh/configs/*.conf	switchable
	TOTP Google Authenticator	pam_google_authenticator	modes B & C
	Verrou compte (faillock)	ssh/sshd_pam	5 → 30 min
	Durcissement sshd (AllowUsers, MaxAuthTries, LoginGraceTime)	ssh/sshd_config	appliqué
Web	Rate-limit nginx (limit_req zone)	nginx/default.conf	5r/min burst=3
	Rate-limit applicatif progressif	login-api/app.py	5s × N, max 60s
	Lockout multi-paliers	login-api/app.py	3 / 5 / 8 échecs
	MFA TOTP (1 tentative)	login-api/app.py (pyotp)	session revoke on fail
	reCAPTCHA v2	login-api/app.py	vérif côté serveur
	Headers de sécurité	nginx/default.conf	XFO, NOSNIFF, Referrer-Policy
Observabilité	Parser SSH (40+ patterns, burst aggregator)	ssh-log-parser/parser.py	anti-bruit
	Filebeat → Elasticsearch 2 indices	filebeat/filebeat.yml	if27-ssh-* / if27-web-*
	Dashboards Kibana	kibana/dashboards.ndjson	2 dashboards
Mitigation active	Détecteur autonome (4 règles)	detector/detector.py	ban iptables 1h
	Honeypot tarpit endlessh	docker-compose.yml (profil)	optionnel

4

Couche SSH — détails

4.1 Trois modes d'authentification

Le projet propose un mécanisme de bascule entre 3 modes pour permettre tant des campagnes de brute-force pédagogiques (mode A) que des configurations durcies proches d'un environnement de production (modes B et C).

Mode	Auth	Cas d'usage
A	password seul	Démonstration d'attaque hydra/ncrack
B	password + TOTP (PAM)	Web app interactive type SSO entreprise
C	clé publique + TOTP (PAM)	Configuration production recommandée

```
./ssh/switch_mode.sh A      # ou B / C – rebuild auto du container
```

4.2 Durcissement sshd

Paramètres communs (dans `ssh/sshd_config`) :

Directive	Valeur	Effet
<code>PermitRootLogin</code>	no	Élimine la cible la plus probable
<code>AllowUsers</code>	admin, testuser	Liste blanche d'utilisateurs autorisés
<code>MaxAuthTries</code>	3	Coupe la session après 3 échecs
<code>LoginGraceTime</code>	20s	Évite les sockets ouvertes par les scanners
<code>PermitEmptyPasswords</code>	no	Défense contre les configs erronées
<code>LogLevel</code>	VERBOSE	Active le logging des clés publiques échouées

4.3 Verrou de compte (pam_faillock)

Ajouté en v2 au PAM stack : après **5 échecs consécutifs**, le compte est verrouillé pendant **30 minutes**, indépendamment de l'IP d'origine. Protège contre :

- Brute-force distribué (botnet, IPs rotatives) qui contourne les bans par IP.
- Credential stuffing depuis des IPs jamais vues.

```
auth required pam_faillock.so preauth silent deny=5 unlock_time=1800
auth [success=1 default=ignore] pam_unix.so nullok
auth [default=die] pam_faillock.so authfail deny=5 unlock_time=1800
auth required pam_google_authenticator.so
```

4.4 Rate-limit iptables (pré-auth)

Au démarrage du container SSH, l'`entrypoint.sh` installe une règle hashlimit qui n'autorise que **3 nouvelles connexions par minute par IP source** sur le port 2222 — avant même que `sshd` ne traite le paquet. Casse les brute-force naïfs sans aucune charge serveur.

4.5 Threat intel Spamhaus DROP

Au boot, l'entrypoint charge la liste publique spamhaus.org/drop/drop.txt (IPs et plages CIDR connues comme malveillantes : botnets, C&C, hébergeurs abusifs) et les bloque préventivement. Couvre ~1500 plages. Réduit le bruit de 30 à 50 %.

5 Couche Web — détails

5.1 Rate-limit nginx (couche 7)

```
limit_req_zone $binary_remote_addr zone=login_limit:10m rate=5r/m;

location /api/login {
    limit_req zone=login_limit burst=3 nodelay;
    limit_req_status 429;
    proxy_pass http://login-api:5000/login;
}
```

5 requêtes / minute / IP avec une rafale autorisée de 3. Tout dépassement renvoie un **HTTP 429 Too Many Requests** sans atteindre l'application.

5.2 Rate-limit applicatif progressif

L'application Flask applique un délai croissant :

Échecs IP	Délai imposé
1-2	aucun
3	15 s
5	25 s
≥ 12	60 s (plafond)

5.3 Lockout multi-paliers

Échecs compte	Niveau	Durée
3	soft_lock_5min	5 minutes
5	hard_lock_15min	15 minutes
8	aggressive_lock_1h	1 heure

5.4 MFA TOTP — une seule tentative

Après validation du mot de passe, l'utilisateur reçoit une session `pending_mfa` et doit fournir un code TOTP (RFC 6238) généré par `pyotp`. La session n'autorise **qu'une tentative** : un code erroné révoque la session pending et oblige à recommencer l'authentification.

5.5 Récupération de compte

Endpoint `/recover` qui génère un token signé de **10 minutes** (en démonstration retourné directement ; en production il serait envoyé par mail). Le reset via `/recover/reset` remet à zéro les compteurs.

5.6 reCAPTCHA v2 + en-têtes de sécurité

- **reCAPTCHA** vérifié serveur via l'API Google avant toute logique métier.

- **X-Frame-Options** : SAMEORIGIN — anti clickjacking.
- **X-Content-Type-Options** : nosniff — anti MIME confusion.
- **Referrer-Policy** : strict-origin-when-cross-origin.
- **Permissions-Policy** : geolocation=(), microphone=(), camera=().

6

Couche observabilité

6.1 Parser SSH custom

Le service `ssh-log-parser` tail `auth.log` et convertit chaque ligne en événement JSON enrichi. Couverture exhaustive :

- **Auth** : succès et échecs (password, publickey, keyboard-interactive, none).
- **PAM** : pam_unix, pam_google_authenticator, pam_faillock.
- **Sessions** : ouverture/fermeture, systemd-logind.
- **Connexions** : open/close/reset, distinction authenticating / invalid.
- **Scanners** : bad protocol, kex errors, banner invalid, refused.
- **Sudo** : commandes et échecs.

6.2 Agrégateur de bursts (rate-limit anti-bruit)

Pour éviter de saturer Elasticsearch avec 50 lignes JSON identiques pendant un brute-force, le parser regroupe en mémoire les événements identiques de même (`IP`, `event_type`, `username`) et émet :

- La **première occurrence** en clair, telle quelle.
- Un **événement de synthèse** `ssh_burst_summary` après 30 s d'inactivité (ou 120 s maximum) qui contient : `burst_count`, `burst_first_seen`, `burst_last_seen`, `burst_rate_per_min`, `risk_level` (critique si ≥ 20).

6.3 Pipeline Filebeat → Elasticsearch → Kibana

- **Filebeat** consomme deux sources de logs JSON et route vers : `if27-web-YYYY.MM.DD` et `if27-ssh-YYYY.MM.DD`.
- **Elasticsearch 7.17** stocke et indexe.
- **Kibana** expose deux dashboards prêts à importer (4 KPIs + timeline + pie + top IPs + top usernames chacun).

7

Mitigation active — la boucle fermée

Le service `ifsecurity-detector` est le bras armé de la plateforme. Il ferme la boucle :

Détection (Elasticsearch) → **Décision** (4 règles) → **Action** (iptables DROP via Docker socket) → **Audit** (bans.log JSON) → **Levée** (TTL 1h)

7.1 Règles de bannissement

Règle	Source	Seuil	Décision
ssh_burst_summary	if27-ssh-*	burst_count ≥ 10	ban immédiat
ssh_failed_password	if27-ssh-*	≥ 8 par IP / 120 s	ban
ssh_invalid_user	if27-ssh-*	≥ 5 par IP / 120 s	ban
web critical risk_level	if27-web-*	tout événement critical	ban

7.2 Application du bannissement

Le détecteur monte `/var/run/docker.sock` en lecture seule. Pour chaque IP fautive, il :

1. Vérifie que l'IP n'est pas déjà bannie (idempotence).
2. Exécute `iptables -A INPUT -s <IP> -j DROP` dans le container `ssh` ET dans `nginx` (les deux surfaces attaquables).
3. Enregistre `until = now + 3600` en mémoire.
4. Trace l'action dans `logs/bans.log` (JSON Lines auditable).
5. À chaque cycle, vérifie les bans expirés et les retire automatiquement.

7.3 Honeypot endlessh (optionnel)

Service activable par profil : `docker compose --profile honeypot up -d endlessh`. Écoute sur le port 22 (réel) et envoie une bannière SSH à 1 octet/seconde. Les scanners et hydra y restent collés plusieurs minutes par tentative. Métrique de soutenance : « temps perdu par l'attaquant ».

8 Couverture des menaces

Menace	Mécanisme(s) qui la couvrent	Couverture
Brute-force SSH single-IP	iptables hashlimit, MaxAuthTries, faillock, détecteur	multi-couche
Brute-force SSH distribué (botnet)	pam_faillock (par compte), Spamhaus DROP, détecteur	couvert
Brute-force web POST /login	nginx limit_req, applicatif progressif, lockout, reCAPTCHA	multi-couche
Credential stuffing	Lockout par compte, MFA, détecteur	couvert
Bypass MFA par session leak	MFA 1-tentative + révocation session si échec	couvert
Reconnaissance / scans pré-auth	iptables hashlimit, Spamhaus DROP, honeypot	partiel
Attaques applicatives OWASP (SQLi/XSS)	Hors scope brute-force	à ajouter
Compromission de l'hôte Docker	Hors scope projet	à ajouter

9.1 Scénario de démonstration

1. **État initial** : ouvrir les dashboards Kibana, montrer 0 attaque.

2. **Lancer une attaque** :

```
hydra -l admin -P rockyou.txt ssh://<host>:2222
```

3. **Détection** : en moins de 10 s, le KPI *Failed auth* s'envole, la timeline montre un pic, le compteur *Burst summaries* grimpe.

4. **Mitigation** : dans la fenêtre de logs du détecteur, l'IP attaquante apparaît avec mention **BAN**.

5. **Vérification** : la commande `docker exec ssh iptables -L INPUT -n` montre la règle DROP. La tentative `hydra` suivante échoue par timeout TCP.

6. **Levée automatique** : après 1 h (paramétrable), le détecteur retire la règle, l'IP redevient accessible.

9.2 Limites connues et perspectives

- **Socket Docker** monté dans le détecteur : acceptable pour un PFE, à remplacer par une API gRPC/REST authentifiée en production.
- **iptables flush** au redémarrage du container : les bans sont reconstitués au cycle suivant du détecteur depuis Elasticsearch.
- **Pas d'IDS réseau** de signatures : perspective d'ajouter Suricata en mode IDS passif pour détecter les outils connus (Nmap, hydra) au niveau paquet.
- **Faux positifs** sur l'IP de l'administrateur : prévoir une whitelist en haut de la chaîne iptables.

IfSecurity démontre qu'une approche **défense en profondeur** orientée mitigation, articulée autour d'un détecteur autonome qui consomme des événements structurés et applique des actions réelles, peut être déployée sur une stack open-source standard (Docker + ELK + Python) sans appliances commerciales. Le projet ferme la boucle entre l'observabilité et la réponse — c'est ce qui distingue une plateforme de défense d'un simple système de monitoring.